Programming 3A

Student Number:	10433940
Programme Code:	BCAD0701
Module Lecturer:	Mortda Adam
Module Code:	PROG7311
Date of Submission:	27-03-2026

I hereby declare that I did not plagiarise the content of this assignment and that this is my own work. I also declare that no use of GenAI was used in the creation of this submission (unless explicitly permitted and declared), excluding grammar and spell checkers. I consent to the lecturer assessing our work using an AI detection tool to ensure the integrity of our work.

Assignment submitted via SafeAssign: ☒ (Tick the Box)

Table of Contents

1 Question 1 1

2 Question 2 3

3 Question 3 7

4 References 9

Total word count excluding headings, in-text references, reference list, diagrams, and word count: 1534

1 QUESTION 1

Introduction

The Global Logistics Management System (GLMS) is an enterprise-grade web platform (Wisdom Library, 2025). The system is designed to assist TechMove Logistics in effectively managing shipping operations. The organisation currently relies on an outdated system of fragmented spreadsheets, emails, and manual phone calls, which has resulted in inconsistent data, misplaced invoices, contract compliance issues, and general operational inefficiencies. The current system has therefore proven to be inefficient, which has highlighted the need for a new system. The new system will make use of architectural framework, design patterns, as well as features to ensure scalability, availability and interoperability.

Executive Summary

The GLMS will replace these disconnected processes and provide a centralised platform for managing key business functions, including contract management for storing legal agreements, client details, and service level agreements (SLAs), service request processing for raising service requests, and financial operations, which will handle international currency conversions. The GLMS is intended for use by the internal staff of TechMove Logistics, such as logistics managers, finance teams, and administrative staff. Other audiences, such as IT staff, would be responsible for system integration, maintenance, and security. The GLMS is a scalable solution that will accommodate the growth of the business. The application will evolve from a monolithic architecture into a cloud-native, service-oriented architecture with containerised deployment using Docker. Through the use of automated testing, integration, and robust API endpoints, the application will ensure maintainability, high availability, and seamless integration with any existing or future systems. [Summary and introduction: 237]

Architectural Framework Strategy

The Zachman Framework is an architectural framework that, according to Ardoq (2024), “provides a formal and structured way of defining and analysing an organisation's information infrastructure.” The Zachman framework consists of rows (perspectives) and columns (interrogatives) (Ardoq, 2024).

For this system, the planner and designer perspectives are applied, along with data, function, and network interrogatives. The planner row under each interrogative defines, in this order, what data exists in the business, what processes and workflows the system will perform, and where the system will be used. In the designer row, the same interrogatives are used but on a more technical level (Ardoq, 2024). This row defines how the data is structured, how the system will work technically, and where the system will run technically. This way, the complex system architecture can be broken down into manageable components (Ardoq, 2024). Additionally, by defining what needs to be

documented and from different perspectives, miscommunication is reduced, improving communication among stakeholders (Capstera, 2023). [152]

Zachman Table

	Data (What)	Function (How)	Network (where)
Planner	Client data, contracts, service requests, invoices, and driver schedules	Contract management, service request processing, validation, and financial handling	Web-based system on a distributed infrastructure supporting scalability and future cloud-native deployment
Designer	SQL Server relational database (Client, Contract, ServiceRequest, Invoice with relationships)	ASP.NET MVC, REST APIs, validation rules, business logic, and external API integration	Initially 3-tier MVC architecture, later evolved into a service-oriented architecture using Docker containers

2 QUESTION 2

Design Pattern Justification

The three Gang of Four (GoF) design patterns that have been selected to implement in code are the Factory Method Pattern (Creational), Observer Pattern (Behavioural), and Mediator Pattern (Behavioural) (GeeksforGeeks, 2015; GeeksforGeeks, 2016; GeeksforGeeks, 2023).

The Factory, according to Tripathi (2024), “it’s ability to decouple the creation of an object from one particular implementation.” This will ensure that code is not bound tightly to specific classes to improve flexibility, as new types of objects can be introduced to a system without modifying existing code, maintenance as changes to objects can be made in a single location, providing overall centralised control over object creation (GeeksforGeeks, 2015; Tripathi, 2024; Mridul, 2023).

The observer pattern is frequently used when multiple objects need to be notified about changes automatically (GeeksforGeeks, 2016). With the use of this pattern, multiple observers can be notified by a subject about state changes, thus ensuring timely updates, which is particularly important to the GLMS (Glushenkov, 2023; Madhavan, 2024). For example, in terms of the GLMS, if the status of the shipment changes from in transit to delivered, the observers, such as the reporting modules, are automatically notified and can react accordingly. Observers can also be used for validation, which the GLMS needs to ensure accurate data capturing (Rafie, 2016).

The Mediator pattern, according to GeeksforGeeks (2017), “centralises communication between objects through a mediator object. One of the main benefits of the mediator is that it promotes decoupling between elements of a system (Project Management Institute, 2018). The mediator will ensure that various components in the GLMS can interact cohesively and changes can be made to components without breaking others (Project Management Institute, 2018). With the use of a mediator, components of the system can interact indirectly, which makes the system easier to maintain, reduces bugs, and supports scalability (Khrienov, 2025) [275]

Factory

Classes/Interfaces

The factory method pattern will use the following classes and interfaces: IContract (interface defining common behaviour for all contracts), DomesticContract, InternationalContract, and PremiumContract (concrete classes for implementation). ContractFactory (factory which decides which concrete contract to create).Client class will contain client details. ContractController will be used.

Relationships

All contract types implement IContract, the factory will create the correct subtype based on the input, and the controller will depend on factory abstraction and not on concrete classes (decoupling).

Dependency Injection

IContractFactory will be injected into the controller or service through a constructor, ensuring the controller depends on abstractions and not the concrete classes.

Event Trigger:

When a user creates a new contract.

Observer

Classes and Interfaces:

ISubject (interface for any entity with an observed state), IObserver (interface for any class that reacts to state changes), ContractStatusSubject, ServiceRequestSubject (concrete subjects tracking state), ComplianceObserver, NotificationObserver, AuditObserver, ReportObserver, IVoiceObserver (concrete observers that respond to changes; observers can be added at run time without changing the subject code).

Relationships:

A subject will maintain a list of observers. Observers can register and unregister themselves. When the state of the subject changes, it will call the appropriate module, and all observers will react automatically.

Dependencies

Observers can be injected into the subject via the program.cs file or through a notification service. The subject maintains a list of IObserver objects. Observers here can be added, replaced or modified.

Event triggers:

Triggered when the subject state changes, observers automatically perform the necessary actions.

Mediator Pattern

IMediator (interface for defining mediator operations), GLMSMediator (concrete mediator implementation). Modules: ContractModule, ServiceModuel, InvoiceModule, CurrencyConversionModule, ScheduleModule, ClientModule, NotificationModule. These Modules communicate through the mediator only.

Relationships

Modules do not communicate directly as the mediator acts as the central coordinator, deciding which module reacts to which event.

Dependency Injection

The IMediator will be injected into each module. The mediator itself can receive services through dependency injections. This ensures that modules are loosely coupled.

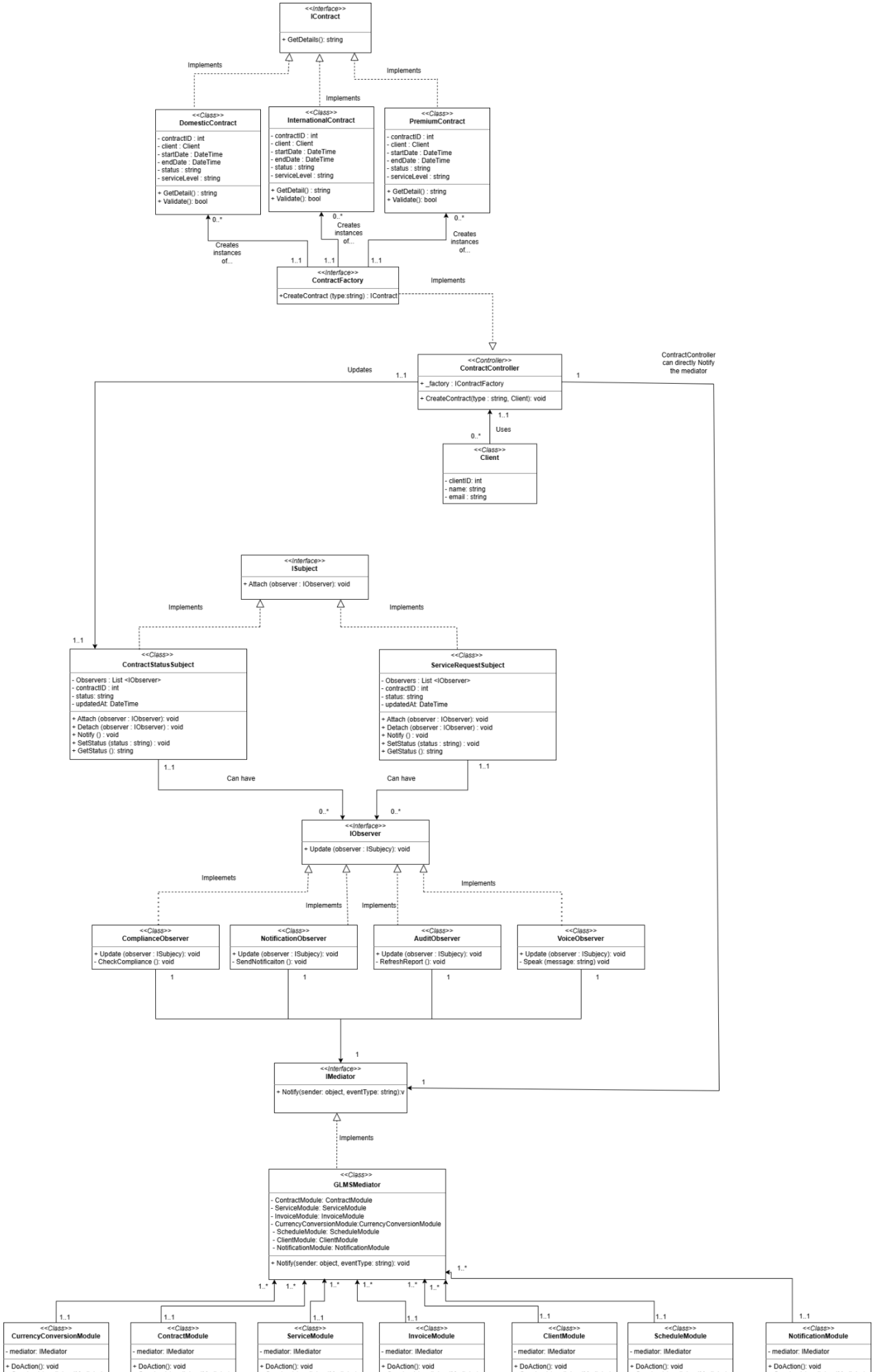
Event Trigger

Different Actions can trigger an event, such as service request submissions, contract approval, and currency conversion. Modules will send these events to the mediator, which will decide the workflow.

[339]

10433940

UML Diagram



3 QUESTION 3

System Availability refers to how often a system is operational and accessible to users (GeeksforGeeks, 2023b). The system should strive to operate continuously with minimal downtime (Flinders and Smalley, 2024). To achieve this, the system will implement the use of robust error handling and logging mechanisms to detect and resolve failures. The system will have no single points of failure (Sheldon, Lutkevich, and Gillis, 2024). The system will support failover mechanisms, ensuring that if one server fails, the other can take over (OVHcloud, 2020). The system will also ensure load balancing to distribute user requests, preventing crashes (Patino, 2025).

System interoperability, according to Digital Adoption (2025), “is the ability of different systems, applications, or devices to communicate and work together without manual adjustments.” The system must be able to integrate with payment systems, external services, and external data sources for currency conversions. To ensure this, the system will make use of Application Programming Interfaces (APIs) for communication by exposing Representational State Transfer Application (RESTful) API endpoints to ensure external systems can interact with the system (Valiente, 2024). APIs will also be used to ensure currency conversion. The system will use JSON formatting for data exchange. (Shaikh, 2024). Interfaces will be used to allow easy integration of external systems by allowing programs to share information with each other (Smith, 2018). These requirements will ensure that the system can communicate and integrate effectively with external systems.

[211]

<u>Scalability</u>

Scalability refers to the ability of a system to handle an increase in workload without compromising performance (Żyliński, 2025). To ensure the system is scalable and can handle a surge of 50,000 applications, the system will make use of Horizontal scaling, load balancing, and caching. Instead of increasing the capacity of the system by upgrading existing services by scaling vertically, TechMove can increase the capacity of the system by adding more servers to distribute the workload across multiple servers, also known as horizontal scaling (GeeksforGeeks, 2021). To allow for horizontal scaling, APIs can be designed to be stateless, meaning they do not store data requests or retain information between sessions (Akamai, 2025). This scaling will ensure the application can accommodate the application surge. Load balancing is the process of distributing application workloads across multiple servers (IBM, 2021). TechMove can implement the dynamic Least Connections Method load balancing algorithm, which is a technique that routes requests to servers with the fewest active connections, focusing on balancing workloads across servers according to workload (GeeksforGeeks, 2024). This ensures loads are balanced effectively, directing traffic to the server that is least busy (GeeksforGeeks, 2024). The Least Response Time Method algorithm can also be used to ensure that requests are sent to the server with the quickest response time, ensuring response time is minimised, allowing the system to process the contracts quicker (GeeksforGeeks, 2024). Caching, according to HazelCast (2024), “is a technique that

allows computer applications to temporarily store data in a computer's memory for quick retrieval.” Redis caching can increase an application’s performance by storing frequently accessed data in memory, reducing response time, resulting in faster runtime (GeeksforGeeks, 2023c). Memcached can also be used, which stores data as basic value pairs, simpler than Redis but effective for fetching and displaying data in read-heavy systems (Dalal, 2025). These caching layers will ensure that the system can function while contracts are being uploaded. [320]

4 REFERENCES

Akamai, 2025. *What Is Horizontal Scaling vs. Vertical Scaling?* [online] Akamai. Available at: <https://www.akamai.com/glossary/what-is-horizontal-scaling-vs-vertical-scaling> [Accessed March 2026].

Ardoq, 2024. *What is the Zachman Framework? A Definitive Guide to this EA Standard*. [online] Ardoq. Available at: <https://www.ardoq.com/knowledge-hub/zachman-framework> [Accessed March 2026].

Campbell, J., 2024. *Zachman Framework*. [online] EBSCO. Available at: <https://www.ebsco.com/research-starters/computer-science/zachman-framework> [Accessed March 2026].

Dalal, D., 2025. *Caching Layers Explained: CDN, Redis, and Memcached — Why They Matter*. [online] Linkedin. Available at: <https://www.linkedin.com/pulse/caching-layers-explained-cdn-redis-memcached-why-matter-deepak-dalal-i5z0c/> [Accessed March 2026].

Flinders, M. and Smalley, I., 2024. *What is High Availability?* [online] IBM. Available at: <https://www.ibm.com/think/topics/high-availability> [Accessed March 2026].

GeeksforGeeks, 2015. *Factory method Design Pattern*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/factory-method-for-designing-pattern/> [Accessed March 2026].

GeeksforGeeks, 2016. *Observer Design Pattern*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/observer-pattern-set-1-introduction/> [Accessed March 2026].

GeeksforGeeks, 2023a. *Availability in System Design*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/availability-in-system-design/> [Accessed March 2026].

GeeksforGeeks, 2023b. *Gang of Four (GOF) Design Patterns*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/gang-of-four-gof-design-patterns/> [Accessed March 2026].

GeeksforGeeks, 2023c. *Redis Cache*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/redis-cache/> [Accessed March 2026].

GeeksforGeeks, 2024. *Load Balancing Algorithms*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/load-balancing-algorithms/> [Accessed March 2026].

Glushenkov, A., 2023. *Notifying the Masses: The Benefits and Challenges of the Observer Pattern in Software Development*. [online] Medium. Available at: <https://medium.com/@alexglushenkov/notifying-the-masses-the-benefits-and-challenges-of-the-observer-pattern-in-software-development-5decd1c008e3> [Accessed March 2026].

HazelCast, 2024. *Caching*. [online] HazelCast. Available at: <https://hazelcast.com/foundations/caching/caching/> [Accessed March 2026].

IBM, 2021. *Load balancing*. [online] IBM. Available at: <https://www.ibm.com/think/topics/load-balancing> [Accessed March 2026].

Khrienov, A., 2025. *The Mediator Pattern: Simplifying Complex Object Interactions in JavaScript*. [online] Medium. Available at: <https://medium.com/@artemkhrenov/the-mediator-pattern-simplifying-complex-object-interactions-in-javascript-698bea5d757d> [Accessed March 2026].

Madhavan, S., 2024. *Real-Time Notifications with the Observer Pattern: A Practical Guide*. [online] DEV Community. Available at: <https://dev.to/sudharshan06/real-time-notifications-with-the-observer-pattern-a-practical-guide-51gi> [Accessed March 2026].

Mediator Design Pattern, 2017. *Mediator Design Pattern*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/system-design/mediator-design-pattern/> [Accessed March 2026].

Mridul, S.S., 2023. *The factory design pattern*. [online] Medium. Available at: <https://medium.com/design-bootcamp/the-factory-design-pattern-ba150444c8a7> [Accessed March 2026].

OVHcloud, 2020. *What is High Availability?* [online] OVHcloud. Available at: <https://www.ovhcloud.com/en-in/learn/what-is-high-availability/> [Accessed March 2026].

Patino, A., 2025. *What is high availability?* [online] Aerospike. Available at: <https://aerospike.com/blog/what-is-high-availability/> [Accessed March 2026].

Project Management Institute, 2018. *The Mediator Pattern*. [online] Project Management Institute . Available at: <https://www.pmi.org/disciplined-agile/the-design-patterns-repository/the-mediator-pattern> [Accessed March 2026].

Rafie, Y., 2016. *Quick Tip: The Convenient Magic of Eloquent Observers*. [online] Sitepoint. Available at: <https://www.sitepoint.com/quick-tip-the-convenient-magic-of-eloquent-observers/> [Accessed March 2026].

Shaikh, R.H., 2024. *Data Interoperability: Principles, Challenges, Best Practices*. [online] Acceldata. Available at: <https://www.acceldata.io/blog/data-interoperability-key-principles-challenges-and-best-practices> [Accessed March 2026].

Sheldon, R., Lutkevich, B. and Gillis, A.S., 2024. *What is high availability (HA)? Definition and guide*. [online] TechTarget. Available at: <https://www.techtarget.com/searchdatacenter/definition/high-availability> [Accessed March 2026].

Smith, G., 2018. *Interface, Interoperability, Integration—Quick Guide for Distributors*. [online] Computers Unlimited. Available at: <https://www.cu.net/industrial/blog/integration-interface-interoperability> [Accessed March 2026].

Valiente, D., 2024. *Understanding APIs: The Backbone of Modern Security Interoperability*. [online] Security Industry Association. Available at: <https://www.securityindustry.org/2024/06/13/understanding-apis-the-backbone-of-modern-security-interoperability/> [Accessed March 2026].

Wisdom Library , 2025. *Global logistics system: Significance and symbolism*. [online] Wisdom Library. Available at: <https://www.wisdomlib.org/concept/global-logistics-system#google_vignette> [Accessed March 2026].

Żyliński, B., 2025. *What Is Scalability - Theory, Tools and How To Scale*. [online] Pask Software. Available at: <https://pasksoftware.com/scalability/> [Accessed March 2026].